

Component Template

Page Content

- [Overview](#)
 - [Concepts](#)
 - [Architecture](#)
 - [Implementation](#)
- [Usage](#)
 - [Declaration of Component in CMake](#)
 - [Configuration of Component in system_config.h](#)
 - [Instantiation and Configuration in CAmkES](#)
 - [Declaring the Component](#)
 - [Connecting a Component](#)
 - [Instantiating and Connecting the Component](#)
 - [Configuring the Instance](#)
- [Example](#)
 - [Instantiation of Component in CMake](#)
 - [Configuration of Component in system_config.h](#)
 - [Instantiation and Configuration in CAmkES](#)
 - [Using the Component's Interfaces in C](#)

Overview

What is the purpose of this component; just a short text.

Concepts

(Only if component has new concepts that need to be understood in order to use it)

Architecture

(Only if component is sufficiently complex and interacts with other parts of the system)

Implementation

No details, just a few generic points how it works internally.

Usage

This is how the component can be instantiated in the system.

Declaration of Component in CMake

Some short text about how to declare the component in the CMakeLists.txt, whether it needs a XXX_client as well and what the params of the DeclareCAmkESComponent_XXX() are:

<Component> Usage 1

```
DeclareCAmkESComponent_XXX(  
    <NameOfComponent>  
)
```

Configuration of Component in system_config.h

Some short text about the #defines to be set here and what they mean:

<Component> Usage 2

```
#define XXX_PARAM_1 10
#define XXX_PARAM_2 11
```

Instantiation and Configuration in CAMkES

Some short text about CAMkES setup:

Declaring the Component

<Component> Usage 3

```
#include "XXX/XXX.camkes"
DECLARE_COMPONENT_XXX(<ComponentTypeName>, ...)
```

Component should provide this macro, even if there are no parameters. This leave the user the freedom to define one or more type name and don't enforce anything.

Connecting a Component

<Component> Usage 4

```
#include "XXX/XXX.camkes"
component <ComponentTypeName> <ComponentInstanceName>
CONNECT_INSTANCE_XXX(<ComponentInstanceName>, ...)
```

Convenience macro to connect the standard interface of a component instance.

Instantiating and Connecting the Component

<Component> Usage 5

```
#include "XXX/XXX.camkes"
DECLARE_AND_CONNECT_INSTANCE_XXX(<ComponentTypeName>, <ComponentInstanceName>, ...)
```

Convenience macro to create a component instance and connect the default interfaces. The recommendation is not to provide this macro unless there is a real benefit here. The CAMkES code is usually easier to understand if component instances are created explicitly and the macro **CONNECT_INSTANCE_XXX()** described above is used.

Configuring the Instance

<Component> Usage 6

```
CONFIGURE_INSTANCE_XXX(<NameOfInstance>, ...)
```

Example

Describe example for the usage for all the relevant parts.

Instantiation of Component in CMake

See above, just with concrete names and parameters.

Configuration of Component in system_config.h

See above, just with concrete names and parameters.

Instantiation and Configuration in CAmkES

See above, just with concrete names and parameters.

Using the Component's Interfaces in C

Maybe add some minimal code examples, e.g. highlighting how to use client-side libraries. But keep it minimal!